

状态管理器（Providers）说明文档

一、概述

在本视频管理应用中，采用 `Provider` 作为状态管理方案，通过 `MultiProvider` 注册了两个核心状态管理器：`VideoProvider`（视频状态管理）和 `TagProvider`（标签状态管理）。这些状态管理器封装了数据存储、业务逻辑和状态更新方法，实现了全应用范围内的数据共享和同步。

二、VideoProvider（视频状态管理器）

1. 核心状态变量

变量名	类型	说明	可访问性
<code>_videos</code>	<code>List<Video></code>	存储所有视频数据的列表（按上传时间倒序）	私有（通过 <code>videos</code> 访问）
<code>_uploadProgress</code>	<code>double</code>	当前视频上传进度（0.0 ~ 1.0）	私有（通过 <code>uploadProgress</code> 访问）
<code>_isUploading</code>	<code>bool</code>	是否正在上传视频（ <code>true</code> 表示上传中）	私有（通过 <code>isUploading</code> 访问）

2. 可访问的属性（供外部组件调用）

属性名	类型	说明
<code>videos</code>	<code>List<Video></code>	获取所有视频列表（只读）
<code>uploadProgress</code>	<code>double</code>	获取当前上传进度（只读）
<code>isUploading</code>	<code>bool</code>	

		获取当前是否正在上传（只读）
--	--	----------------

3. 核心方法（业务逻辑）

方法名	参数	返回值	说明
<code>loadVideos()</code>	无	<code>void</code>	从本地数据库加载所有视频，更新 <code>_videos</code> 并通知组件刷新
<code>getVideoById(String id)</code>	<code>id</code> ：视频唯一标识	<code>Video?</code>	根据 ID 查询单个视频（不存在返回 <code>null</code> ）
<code>saveVideo(Video video)</code>	<code>video</code> ：视频对象	<code>Future<void></code>	将视频数据保存到本地数据库，保存后重新加载列表
<code>deleteVideo(String id)</code>	<code>id</code> ：视频唯一标识	<code>Future<void></code>	根据 ID 删除视频，删除后重新加载列表
<code>searchVideos(String keyword)</code>	<code>keyword</code> ：搜索关键词	<code>void</code>	根据关键词（标题、标签、备注）筛选视频，更新列表
<code>updateUploadProgress(double progress)</code>	<code>progress</code> ：进度值（0.0~1.0）	<code>void</code>	更新上传进度，自动设置 <code>_isUploading</code> 状态
<code>resetUploadState()</code>	无	<code>void</code>	重置上传状态（进度归零，标记为未上传）

4. 适用场景

- 首页视频列表展示、搜索筛选
- 视频上传进度展示
- 视频详情页数据获取

- 视频删除 / 编辑操作

三、TagProvider（标签状态管理器）

1. 核心状态变量

变量名	类型	说明	可访问性
<code>_tags</code>	<code>List<Tag></code>	存储所有标签数据的列表（按名称正序）	私有（通过 <code>tags</code> 访问）

2. 可访问的属性（供外部组件调用）

属性名	类型	说明
<code>tags</code>	<code>List<Tag></code>	获取所有标签列表（只读）

3. 核心方法（业务逻辑）

方法名	参数	返回值	说明
<code>loadTags()</code>	无	<code>void</code>	从本地数据库加载所有标签，更新 <code>_tags</code> 并通知组件刷新
<code>getTagById(String id)</code>	<code>id</code> : 标签唯一标识	<code>Tag?</code>	根据 ID 查询单个标签（不存在返回 <code>null</code> ）
<code>createTag(String name)</code>	<code>name</code> : 标签名称	<code>Future<Tag?></code>	创建新标签（名称唯一校验），成功返回标签对象，失败返回 <code>null</code>
<code>updateTag(String id, String newName)</code>	<code>id</code> : 标签 ID; - <code>newName</code> : 新名称	<code>Future<Tag?></code>	更新标签名称（名称唯一校验），成功返回更新后对象，失败返回 <code>null</code>

`deleteTag(String id)`

id: 标签唯一标识		Future<void>	删除标签，并自动移除所有视频中关联的该标签 ID
recalculateVideoCounts()	无	Future<void>	重新计算每个标签关联的视频数量（同步到 videoCount 属性）

4. 适用场景

- 标签管理页的标签列表展示、增删改操作
- 视频上传页的标签选择功能
- 视频列表中标签的显示（如视频卡片上的标签芯片）

四、状态使用方式

在任意子组件中，通过 `Provider.of` 方法获取状态管理器实例，即可访问状态和调用方法：

```
// 示例：在组件中获取视频和标签状态
Widget build(BuildContext context) {
  // 获取视频状态管理器
  final videoProvider = Provider.of<VideoProvider>(context);
  // 获取标签状态管理器
  final tagProvider = Provider.of<TagProvider>(context);
  // 使用视频状态
  List<Video> allVideos = videoProvider.videos;
  bool isUploading = videoProvider.isUploading;
  // 使用标签状态
  List<Tag> allTags = tagProvider.tags;
  Tag? tag = tagProvider.getTagById("tag123");
  return ...;
}
```

五、扩展说明

- **状态更新通知：**当 `Provider` 中的状态（如 `_videos`、`_tags`）发生变化时，需调用 `notifyListeners()` 方法通知所有依赖该状态的组件重新构建（代码中已封装该逻辑）。
- **新增状态：**若需扩展状态（如 “当前播放的视频 ID”），可在对应 `Provider` 中添加变量及更新方法，例如在 `VideoProvider` 中添加 `String? currentPlayingVideoId` 并提供 `setCurrentPlayingVideoId(String id)` 方法。
- **性能优化：**通过 `Consumer` 或 `Selector` 组件可实现局部刷新，避免因状态更新导致的全页面重建（复杂页面推荐使用）。

（注：文档部分内容可能由 AI 生成）